

(Procedimento Operacional Padrão para Extração e Análise de Dados em Banco de Dados aplicado à Engenharia de Produção)

João Paulo Alves da Rocha (Universidade Federal de Goiás)



O avanço da Indústria 4.0 intensificou a geração e o uso de dados nas organizações, tornando a capacidade de análise um fator estratégico para a tomada de decisão. Nesse contexto, a Engenharia de Produção assume papel fundamental na transformação de dados operacionais em informações relevantes para melhoria de processos. Entretanto, muitas empresas ainda dependem de relatórios pré-processados e planilhas, limitando a autonomia analítica dos profissionais. Diante disso, o presente trabalho tem como objetivo desenvolver um Procedimento Operacional Padrão (POP) para acesso, extração e análise de dados diretamente em bancos de dados relacionais. A metodologia adotada envolve a fundamentação teórica sobre sistemas de informação, arquitetura de software, bancos de dados e modelagem de dados, seguida da aplicação prática por meio de consultas estruturadas em linguagem SQL. O estudo aborda desde a organização dos dados até sua utilização na geração de indicadores gerenciais, evidenciando a importância do acesso direto às bases de dados. Como resultado, observa-se que a utilização de bancos de dados relacionais permite maior precisão, flexibilidade e agilidade na análise das informações, contribuindo significativamente para a melhoria da tomada de decisão. Conclui-se que o domínio de ferramentas de acesso a banco de dados representa uma competência essencial para o engenheiro de produção no contexto atual..

Palavras-chave: Banco de Dados; Engenharia de Produção; SQL; Análise de Dados; Sistemas de Informação.

1. Introdução

O avanço da Indústria 4.0 tem intensificado a geração e o uso de dados nas organizações, tornando-os um dos principais ativos estratégicos para a tomada de decisão. Nesse contexto, a Engenharia de Produção assume papel fundamental ao transformar dados operacionais em informações relevantes para melhoria de processos, aumento de produtividade e redução de desperdícios. Segundo Klaus Schwab (2016), a quarta revolução industrial é caracterizada pela integração de sistemas físicos e digitais, ampliando significativamente a capacidade de coleta e análise de dados em tempo real.

Apesar dessa crescente disponibilidade de dados, muitas empresas ainda utilizam métodos tradicionais de análise, baseados em planilhas eletrônicas e relatórios previamente estruturados. Essa abordagem apresenta limitações importantes, como a defasagem temporal das informações, a perda de granularidade dos dados e a dependência de áreas técnicas para geração de novas análises. De acordo com Thomas H. Davenport e Harris (2017), organizações orientadas a dados devem desenvolver autonomia analítica em seus profissionais, permitindo acesso direto às fontes primárias de informação.

Nesse cenário, os bancos de dados se consolidam como a principal fonte de dados confiáveis e atualizados dentro das organizações. Diferentemente das planilhas, que representam apenas recortes da realidade operacional, os sistemas de gerenciamento de banco de dados (SGBDs) armazenam grandes volumes de dados estruturados, com integridade, rastreabilidade e possibilidade de consulta em tempo real (Elmasri; Navathe, 2018). Assim, o acesso direto ao banco de dados possibilita ao engenheiro de produção maior autonomia na construção de análises, reduzindo a dependência de intermediários e aumentando a agilidade na tomada de decisão.

Além disso, a capacidade de extrair e interpretar dados diretamente do banco está diretamente relacionada ao desenvolvimento de competências analíticas essenciais na formação do engenheiro de produção. Conforme destaca Erik Brynjolfsson (2020), o diferencial competitivo das organizações está cada vez mais associado à habilidade de transformar dados em conhecimento aplicável, o que exige domínio de ferramentas tecnológicas e compreensão dos sistemas de informação.

Diante desse contexto, torna-se necessário estruturar procedimentos padronizados que orientem o acesso, a extração e a análise de dados diretamente em bancos de dados. Nesse sentido, o presente trabalho tem como objetivo desenvolver um Procedimento Operacional Padrão (POP), no formato de artigo científico, que descreva de forma sistemática o processo de conexão a um

banco de dados, execução de consultas e análise de informações aplicadas à Engenharia de Produção.

Por fim, este estudo busca demonstrar, na prática, como o acesso direto ao banco de dados contribui para a geração de indicadores relevantes e para o suporte à tomada de decisão, reforçando o papel do engenheiro de produção como agente estratégico na transformação de dados em conhecimento e valor para as organizações.

2. Fundamentação Teórica

Para que seja possível desenvolver um Procedimento Operacional Padrão (POP) voltado à extração e análise de dados em banco de dados, é necessário compreender os principais conceitos que sustentam esse processo dentro das organizações. A Engenharia de Produção, por atuar diretamente na análise e melhoria de processos, depende fortemente de informações confiáveis e atualizadas para apoiar a tomada de decisão.

Nesse contexto, a fundamentação teórica deste trabalho busca apresentar, de forma aplicada, os conceitos essenciais relacionados aos sistemas de informação, arquitetura de software e bancos de dados. Esses elementos são responsáveis por estruturar a forma como os dados são gerados, armazenados e utilizados dentro das empresas.

Ao longo desta seção, serão abordados temas como o papel dos Sistemas de Informação Gerencial (SIG), a organização dos sistemas computacionais em camadas, a evolução dos bancos de dados e os princípios de modelagem de dados. O objetivo não é apenas apresentar conceitos teóricos, mas demonstrar como esses conhecimentos são utilizados na prática, especialmente no contexto da Engenharia de Produção, onde a análise de dados é fundamental para a melhoria contínua dos processos.

2.1 Sistemas de Informação Gerencial (SIG)

Os Sistemas de Informação Gerencial (SIG) são frequentemente confundidos com softwares utilizados nas empresas, porém essa interpretação é limitada. Na realidade, o SIG deve ser compreendido como um processo estruturado de transformação de dados em informações úteis, com o objetivo de apoiar a tomada de decisão dentro das organizações.

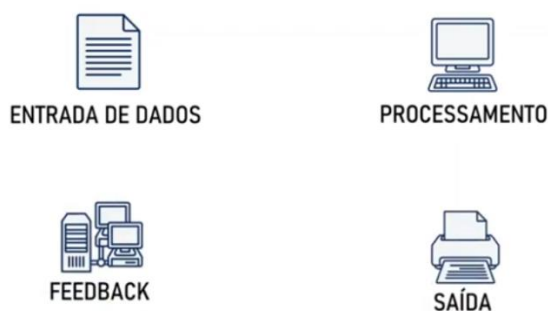
De acordo com Kenneth C. Laudon e Jane P. Laudon (2021), um sistema de informação é composto por elementos inter-relacionados responsáveis por coletar, processar, armazenar e distribuir informações. No entanto, mais do que um conjunto de ferramentas tecnológicas, o SIG representa um fluxo contínuo de transformação, no qual os dados gerados pelas

operações da empresa são convertidos em informações e, posteriormente, em conhecimento aplicado à gestão.

Nesse sentido, é importante destacar que o SIG não se limita a um software específico, como um ERP (Enterprise Resource Planning). O ERP é apenas uma ferramenta que viabiliza o funcionamento do sistema de informação, enquanto o SIG corresponde ao processo mais amplo que envolve pessoas, tecnologias, dados e métodos de análise. Em outras palavras, o ERP registra e organiza os dados, mas é o SIG que estrutura a forma como esses dados são utilizados para gerar valor para a organização.

Para compreender melhor esse conceito, pode-se observar o fluxo apresentado na Figura 1, que ilustra o funcionamento básico de um sistema de informação.

Figura 1 – Processo SIG



Fonte: Elaborado pelos Autores (2026)

Nesse modelo, a entrada de dados corresponde à coleta de informações brutas oriundas das atividades operacionais, como registros de produção, tempos de máquina, quantidades produzidas e perdas de material. Em seguida, ocorre o processamento, no qual esses dados são organizados e analisados, transformando-se em informações estruturadas.

A etapa de saída consiste na apresentação dessas informações, geralmente por meio de relatórios, indicadores ou dashboards, que permitem ao gestor compreender o desempenho dos processos. Por fim, o feedback representa o retorno dessas informações ao sistema, possibilitando ajustes nas operações e melhorias contínuas.

O feedback pode se manifestar de diversas formas dentro da organização. Um dos principais exemplos é o resultado financeiro, especialmente o lucro, que reflete diretamente a eficiência dos processos produtivos e das decisões gerenciais. Quando uma empresa reduz desperdícios, melhora sua produtividade ou otimiza seus recursos com base em análises de dados, essas melhorias tendem a impactar positivamente seus resultados financeiros. Além disso, outros

indicadores, como qualidade, nível de serviço e satisfação do cliente, também funcionam como mecanismos de feedback, orientando ajustes no sistema.

Embora o conceito de SIG não dependa exclusivamente de tecnologia, na prática, sua aplicação nas organizações modernas está diretamente associada ao uso de sistemas computacionais. Isso ocorre porque o volume e a complexidade dos dados gerados atualmente tornam inviável a utilização de métodos manuais, como registros em papel ou controles isolados. Conforme apontam Turban et al. (2020), o uso de sistemas informatizados é essencial para garantir a velocidade, a precisão e a confiabilidade das informações utilizadas na gestão.

Dessa forma, nos dias atuais, a implementação de um Sistema de Informação Gerencial eficiente exige o uso de softwares de gestão empresarial, como ERPs, que permitem registrar, armazenar e acessar grandes volumes de dados de forma integrada. No entanto, é importante reforçar que esses sistemas são meios, e não o fim: o verdadeiro valor está na capacidade de transformar os dados gerados em informações que apoiem decisões estratégicas.

Portanto, o SIG deve ser entendido como um processo dinâmico e contínuo, que conecta a operação à gestão por meio da análise de dados. Para o engenheiro de produção, essa compreensão é essencial, pois permite não apenas utilizar os sistemas existentes, mas também explorar os dados em sua origem, ampliando a capacidade analítica e contribuindo de forma mais efetiva para a melhoria dos processos organizacionais.

2.2 Arquitetura de Software

Para que os Sistemas de Informação Gerencial (SIG) funcionem de forma eficiente dentro das organizações, é necessário compreender como os sistemas computacionais são estruturados e como seus componentes interagem entre si. Essa organização é definida pela arquitetura de software, que estabelece a forma como as diferentes partes de um sistema se comunicam, trocam informações e executam suas funções.

De acordo com Bass, Clements e Kazman (2021), a arquitetura de software define a estrutura de um sistema, seus componentes e as relações entre eles, sendo responsável por garantir que o sistema atenda aos requisitos de desempenho, confiabilidade e escalabilidade. Dessa forma, a arquitetura não está relacionada apenas à interface visível ao usuário, mas principalmente à forma como o sistema funciona internamente.

Um dos modelos mais conhecidos para organizar essa interação entre componentes é o padrão MVC (Model-View-Controller). Esse modelo divide o sistema em três partes principais, cada

uma com uma responsabilidade específica, permitindo maior organização e clareza na estrutura do sistema.

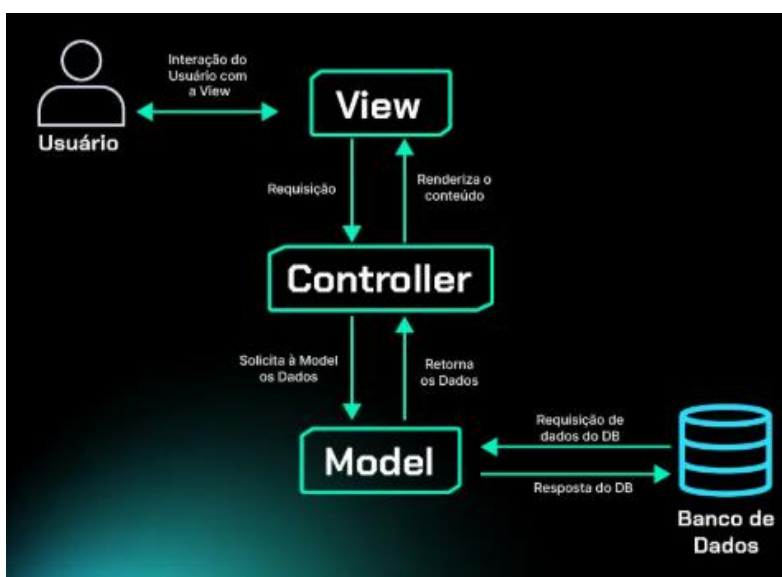
O Model (Modelo) é responsável pela gestão dos dados e pelas regras de negócio, sendo a camada que se comunica diretamente com o banco de dados. É nessa camada que estão armazenadas as informações da empresa, como ordens de produção, registros de estoque e dados operacionais.

A View (Visão) corresponde à interface com o usuário, ou seja, às telas, relatórios e dashboards apresentados no sistema. É por meio dessa camada que o engenheiro de produção acessa informações, insere dados e acompanha indicadores.

O Controller (Controlador) atua como intermediário entre o Model e a View. Ele recebe as ações do usuário, processa essas solicitações e coordena a comunicação entre a interface e os dados, garantindo que o sistema responda corretamente às interações.

A Figura 2 apresenta a estrutura do modelo MVC, evidenciando a interação entre o usuário, o sistema e o banco de dados.

Figura 2 – MVC



Fonte: Alura (2026)

Conforme ilustrado na Figura 2, o usuário interage diretamente com a interface (View), que encaminha as requisições ao Controller. O Controller, por sua vez, solicita os dados ao Model, que realiza a comunicação com o banco de dados. Após o processamento, as informações retornam pelo mesmo fluxo até serem apresentadas ao usuário.

Esse modelo evidencia que o banco de dados não é acessado diretamente pela interface, mas sim por meio de uma estrutura organizada, o que garante maior controle, segurança e padronização no acesso às informações.

Além do modelo MVC, muitos sistemas utilizam uma organização em camadas, que complementa essa estrutura. Essas camadas geralmente são divididas em três níveis principais: interface, lógica de aplicação e banco de dados.

A camada de interface corresponde à interação com o usuário, sendo responsável pela apresentação das informações. A camada de lógica de aplicação concentra as regras do sistema, realizando cálculos, validações e tratamentos dos dados. Já a camada de banco de dados é responsável pelo armazenamento das informações, sendo considerada a fonte primária dos dados organizacionais.

Para ilustrar esse funcionamento, pode-se considerar um exemplo prático de um sistema ERP utilizado em uma indústria. Ao registrar a produção de uma máquina, o operador insere os dados na interface do sistema. Essas informações passam pela lógica de aplicação, onde são validadas e processadas, e, em seguida, são armazenadas no banco de dados. Posteriormente, quando o engenheiro de produção consulta um relatório, o sistema busca esses dados, realiza o processamento necessário e apresenta as informações na tela.

Esse fluxo evidencia que, embora o usuário interaja diretamente com o sistema por meio de telas e relatórios, os dados reais estão armazenados no banco de dados. Assim, ao utilizar apenas a interface do sistema, o usuário acessa uma visão já processada das informações, que pode não conter o nível de detalhe necessário para análises mais aprofundadas.

Nesse contexto, o acesso direto ao banco de dados torna-se uma prática estratégica para o engenheiro de produção, pois permite explorar os dados em sua forma original, realizar consultas personalizadas e desenvolver análises mais precisas. Esse tipo de abordagem reduz a dependência de relatórios padronizados e amplia a capacidade de investigação dos processos produtivos.

Dessa forma, compreender a arquitetura de software, especialmente os modelos de organização como o MVC e a divisão em camadas, é fundamental para identificar onde os dados estão localizados, como são processados e de que maneira podem ser acessados. Esse entendimento é essencial para a construção do Procedimento Operacional Padrão (POP) proposto neste trabalho, uma vez que o acesso ao banco de dados depende diretamente da forma como o sistema foi estruturado.

2.3 Banco de Dados

Os bancos de dados constituem o núcleo dos sistemas de informação nas organizações, sendo responsáveis pelo armazenamento e organização dos dados gerados pelas atividades operacionais. No contexto da Engenharia de Produção, esses dados são essenciais para a

análise de processos, controle de desempenho e suporte à tomada de decisão. No entanto, para compreender plenamente a importância dos bancos de dados na atualidade, é necessário analisar sua evolução ao longo do tempo.

Na década de 1950, antes da popularização dos computadores nas empresas, os dados eram armazenados de forma totalmente manual, por meio de papéis, formulários e pastas físicas. Esse modelo, embora funcional para volumes reduzidos de informação, gerava diversos problemas, como acúmulo excessivo de documentos, dificuldade de organização, risco de perda de informações e baixa eficiência na busca por dados.

Figura 3 – Armazenamento de dados em papel e pastas



Fonte: Curso em Video (2016)

Nesse cenário, localizar uma informação específica exigia a busca manual entre diversos arquivos físicos, tornando o processo lento e sujeito a erros. Esse contexto evidenciava a necessidade de métodos mais eficientes de armazenamento e recuperação de dados.

Com o avanço da tecnologia e a introdução dos computadores nas organizações, iniciou-se a digitalização desses registros. Inicialmente, os dados passaram a ser armazenados em arquivos sequenciais, nos quais as informações eram organizadas de forma linear, semelhante a uma fita. Nesse modelo, para acessar um determinado dado, era necessário percorrer todos os registros anteriores, o que tornava o processo de consulta lento e pouco eficiente.

Posteriormente, com a evolução dos dispositivos de armazenamento, especialmente o uso de discos magnéticos, tornou-se possível o chamado acesso direto aos dados, no qual as informações podiam ser localizadas sem a necessidade de leitura sequencial. Esse avanço representou um ganho significativo em desempenho e abriu caminho para o desenvolvimento dos primeiros sistemas de banco de dados.

Na década de 1960, surgiram os primeiros modelos estruturados de banco de dados, impulsionados principalmente por iniciativas do governo e do exército dos Estados Unidos. Destacam-se, nesse período, os modelos hierárquicos e em rede, desenvolvidos no contexto

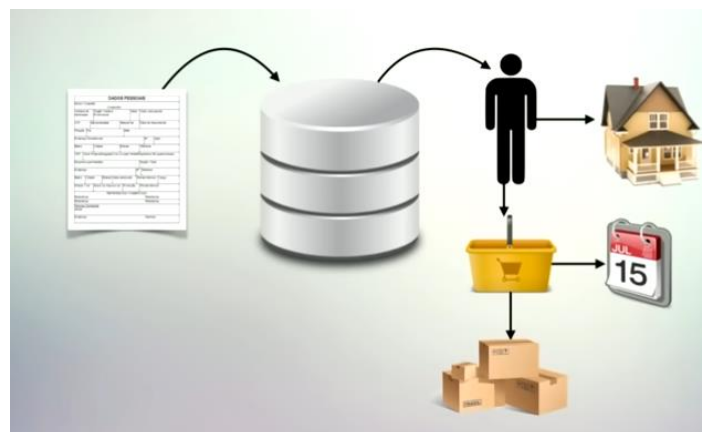
do CODASYL (Conference on Data Systems Languages). O modelo hierárquico organizava os dados em uma estrutura de árvore, na qual cada registro possuía um único elemento pai. Já o modelo em rede permitia que um registro tivesse múltiplas relações, oferecendo maior flexibilidade.

Apesar desses avanços, esses modelos apresentavam limitações significativas, especialmente no que se refere à complexidade de implementação e à dificuldade de representação de relacionamentos mais amplos entre os dados. Isso dificultava a realização de consultas e a adaptação dos sistemas a novas necessidades.

Diante dessas limitações, na década de 1970, o pesquisador Edgar F. Codd propôs o modelo relacional, que revolucionou a forma de organização dos dados. Nesse modelo, as informações passam a ser estruturadas em tabelas (relações), compostas por linhas e colunas, permitindo maior simplicidade, flexibilidade e eficiência na manipulação dos dados.

No modelo relacional, os dados são organizados de forma que seja possível estabelecer relações entre diferentes tabelas. Isso permite, por exemplo, cadastrar um cliente em uma tabela e, a partir dele, acessar diversas outras informações relacionadas, como endereço, histórico de compras e impacto dessas compras no estoque da empresa.

Figura 4 – Exemplo de relacionamento de dados em banco relacional



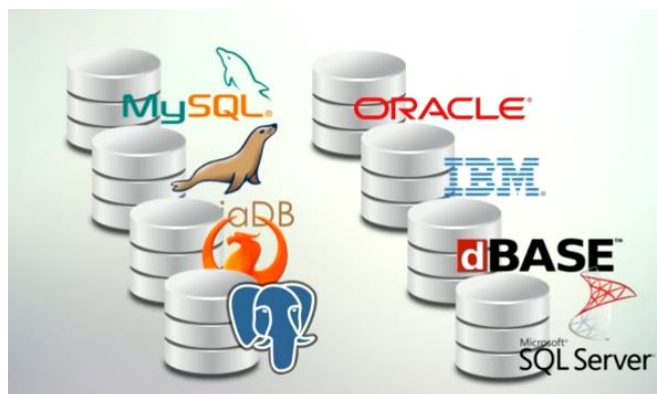
Fonte: Aula em Vídeo (2016)

Conforme ilustrado na Figura 4, um único registro pode estar conectado a diferentes informações, permitindo uma visão integrada dos dados organizacionais. No contexto da Engenharia de Produção, isso possibilita análises mais completas, como identificar quais produtos um cliente consome, qual o impacto dessas vendas na produção e como isso afeta o planejamento de materiais.

Para realizar consultas nesses bancos de dados, utiliza-se a linguagem SQL (Structured Query Language), que se tornou o padrão para manipulação de dados em sistemas relacionais. A SQL permite executar operações como consulta, inserção, atualização e exclusão de dados de forma estruturada e eficiente.

Embora a proposta inicial fosse a existência de uma única linguagem padrão, na prática surgiram diferentes implementações de SQL, em função da concorrência entre fabricantes de sistemas de banco de dados. Atualmente, existem diversas soluções no mercado, tanto proprietárias quanto gratuitas, como Oracle, SQL Server, MySQL e PostgreSQL, cada uma com suas particularidades e extensões da linguagem SQL.

Figura 5 – Exemplos de sistemas de banco de dados (SGBDs)



Fonte: Aula em Video (2016)

Esses sistemas são amplamente utilizados nas organizações, sendo responsáveis pelo armazenamento de grandes volumes de dados com integridade e segurança. No contexto empresarial, especialmente em sistemas ERP, os bancos relacionais são predominantes, devido à necessidade de consistência e controle das informações.

Com o avanço da tecnologia e o crescimento exponencial do volume de dados, surgiram, a partir dos anos 2000, os bancos de dados não relacionais (NoSQL). Diferentemente do modelo relacional, esses bancos não utilizam estruturas rígidas baseadas em tabelas, permitindo maior flexibilidade na organização dos dados.

Os bancos NoSQL foram desenvolvidos para atender cenários em que os dados possuem alta variabilidade, grande volume ou necessidade de processamento em tempo real. Nesse modelo, os dados podem ser armazenados em formatos como documentos (JSON), pares chave-valor, colunas ou grafos, o que permite maior adaptação às diferentes aplicações.

Na prática, os bancos NoSQL são amplamente utilizados em contextos onde o modelo relacional se torna limitado. Um exemplo comum é o uso em sistemas de Internet das Coisas

(IoT), nos quais sensores coletam dados continuamente, como temperatura, vibração ou tempo de operação de máquinas. Nesse caso, a quantidade de dados gerada é muito grande e não necessariamente segue uma estrutura fixa, tornando os bancos NoSQL mais adequados. Outro exemplo relevante é o uso em sistemas de monitoramento em tempo real, como acompanhamento de desempenho de linhas de produção ou controle de máquinas industriais. Nesses cenários, os dados precisam ser armazenados e processados rapidamente, muitas vezes em tempo real, o que exige alta escalabilidade.

Além disso, bancos NoSQL são amplamente utilizados em aplicações web e plataformas digitais, como sistemas de recomendação, redes sociais e e-commerces, onde há grande volume de acessos simultâneos e necessidade de rápida resposta. Nesses ambientes, a flexibilidade na estrutura dos dados permite armazenar diferentes tipos de informação sem a necessidade de um esquema rígido.

Apesar dessas vantagens, os bancos NoSQL geralmente não seguem o mesmo nível de consistência dos bancos relacionais. Enquanto os bancos relacionais utilizam o modelo ACID (Atomicidade, Consistência, Isolamento e Durabilidade), os bancos NoSQL frequentemente adotam o modelo BASE (Basicamente Disponível, Estado Suave e Eventualmente Consistente), priorizando desempenho e disponibilidade em detrimento da consistência imediata dos dados (SADALAGE; FOWLER, 2013).

Diante dessas diferenças, observa-se que não existe um único modelo de banco de dados ideal para todos os cenários. Essa percepção levou ao surgimento do conceito de persistência poliglota (polyglot persistence), que consiste na utilização de diferentes tipos de bancos de dados dentro de um mesmo sistema, de acordo com a necessidade de cada aplicação.

Segundo Fowler (2012), a persistência poliglota reconhece que diferentes problemas exigem diferentes soluções de armazenamento. Dessa forma, uma organização pode utilizar bancos relacionais para sistemas estruturados, como ERP, controle de produção e financeiro, enquanto utiliza bancos NoSQL para aplicações que demandam alta escalabilidade, como coleta de dados em tempo real ou armazenamento de logs.

No contexto da Engenharia de Produção, essa abordagem pode ser observada, por exemplo, em uma indústria que utiliza um banco relacional para gerenciar ordens de produção, estoque e planejamento, enquanto utiliza um banco NoSQL para armazenar dados de sensores de máquinas ou históricos de operação em grande escala. Essa combinação permite explorar o melhor de cada tecnologia, garantindo tanto a integridade dos dados quanto a capacidade de processamento de grandes volumes de informação.

Dessa forma, enquanto os bancos relacionais continuam sendo fundamentais para sistemas que exigem organização estruturada e consistência dos dados, os bancos não relacionais complementam esse cenário ao oferecer flexibilidade e escalabilidade. A escolha entre esses modelos, ou mesmo a combinação entre eles, depende diretamente das necessidades do sistema e dos objetivos da análise.

No contexto deste trabalho, voltado à Engenharia de Produção, o modelo relacional se destaca como o mais adequado para a análise de dados operacionais, uma vez que os dados são estruturados e possuem relações bem definidas. No entanto, compreender o papel dos bancos NoSQL e da persistência poliglota amplia a visão sobre as possibilidades de armazenamento e análise de dados nas organizações modernas.

2.4 Banco de Dados Relacional (SQL) x Não Relacional (NoSQL)

Os bancos de dados podem ser classificados, de forma geral, em dois grandes grupos: os bancos de dados relacionais (SQL) e os bancos de dados não relacionais (NoSQL). Cada um desses modelos possui características próprias, sendo mais adequado para determinados tipos de aplicação.

Os bancos de dados relacionais, também conhecidos como SQL, são baseados em uma estrutura organizada em tabelas, compostas por linhas e colunas. Nessas tabelas, cada registro representa uma entidade, e os dados são estruturados de forma bem definida, com campos previamente estabelecidos. Além disso, uma das principais características desse modelo é a possibilidade de estabelecer relações entre diferentes tabelas, permitindo integrar informações de maneira consistente.

Por exemplo, em um sistema ERP, é possível ter uma tabela de clientes, outra de pedidos e outra de produtos. Por meio dos relacionamentos, é possível consultar quais produtos um determinado cliente comprou, quando ocorreu a compra e qual foi o impacto dessa venda no estoque. Esse tipo de organização permite análises completas e integradas, sendo amplamente utilizado em sistemas empresariais.

A linguagem utilizada nesses bancos é a SQL (Structured Query Language), considerada um padrão para bancos de dados relacionais. Essa linguagem permite realizar consultas, inserções, atualizações e exclusões de dados de forma estruturada e padronizada. Embora existam variações entre fabricantes, como Oracle, SQL Server, MySQL e PostgreSQL, todas seguem o mesmo conceito básico da linguagem SQL.

Figura 6 – Exemplos de bancos de dados relacionais (SQL)



Fonte: Hashtag Programação (2026)

Por outro lado, os bancos de dados não relacionais (NoSQL) surgiram como uma alternativa ao modelo tradicional, especialmente para atender aplicações que demandam maior flexibilidade e escalabilidade. Diferentemente dos bancos relacionais, o NoSQL não utiliza uma estrutura rígida baseada em tabelas. Os dados são geralmente armazenados em formatos mais flexíveis, como documentos (por exemplo, JSON), permitindo que diferentes registros tenham estruturas distintas.

Nesse modelo, não há a mesma necessidade de definir previamente todos os campos e relações, o que facilita o armazenamento de dados variados e em grande volume. No entanto, essa flexibilidade vem acompanhada de menor padronização na estrutura e na forma de consulta.

Outra diferença importante é que os bancos NoSQL não possuem uma linguagem única padrão como o SQL. Cada sistema pode utilizar sua própria forma de consulta e manipulação de dados, como ocorre no MongoDB, Cassandra ou Neo4j, o que exige maior adaptação por parte do usuário.

Figura 7 – Exemplos de bancos de dados não relacionais (NoSQL)



Fonte: Hashtag Programação (2023)

Na prática, os bancos relacionais são amplamente utilizados em sistemas estruturados, como ERPs, sistemas financeiros, controle de estoque e gestão de produção, onde é fundamental garantir a integridade e o relacionamento entre os dados. Já os bancos NoSQL são mais comuns em aplicações como redes sociais, aplicativos mobile, plataformas de streaming, sistemas de recomendação e monitoramento em tempo real, nos quais há grande volume de dados e necessidade de alta performance.

Por exemplo, um sistema ERP de uma indústria utiliza banco relacional para controlar pedidos, produção e estoque, garantindo que todas as informações estejam integradas e

consistentes. Por outro lado, um aplicativo como uma rede social utiliza banco NoSQL para armazenar posts, interações e dados de usuários, pois esses dados possuem formatos variados e são gerados em grande escala.

Dessa forma, observa-se que os bancos SQL e NoSQL não são concorrentes diretos, mas sim tecnologias complementares, cada uma adequada a diferentes necessidades. No contexto da Engenharia de Produção, o modelo relacional tende a ser mais utilizado, devido à necessidade de organização e relacionamento entre os dados. No entanto, compreender as características dos bancos não relacionais é fundamental para ampliar a capacidade de análise e adaptação a diferentes cenários tecnológicos.

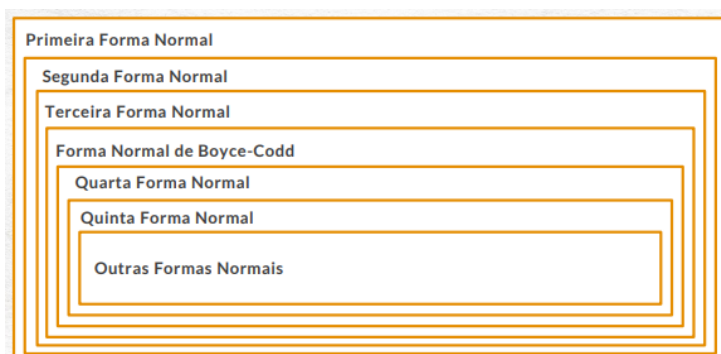
2.5 Normalização dos Dados

A normalização de dados é um processo fundamental na modelagem de bancos de dados relacionais, tendo como principal objetivo garantir que as informações estejam organizadas de forma consistente, evitando redundâncias e inconsistências. Esse processo consiste em aplicar uma série de regras, conhecidas como formas normais, que orientam a estruturação das tabelas e a relação entre os dados.

De maneira geral, a normalização busca corrigir problemas comuns em bancos de dados mal estruturados, como duplicidade de informações, dificuldades de atualização e perda de dados em operações de exclusão. Esses problemas são conhecidos como anomalias de inserção, atualização e exclusão, e impactam diretamente a confiabilidade das informações utilizadas na tomada de decisão.

O processo de normalização é dividido em etapas chamadas de formas normais, que devem ser aplicadas de maneira sequencial. Cada forma normal possui um conjunto de requisitos que precisam ser atendidos antes de avançar para a próxima etapa, garantindo que o banco de dados seja progressivamente organizado.

Figura 8 – Estrutura das Formas Normais

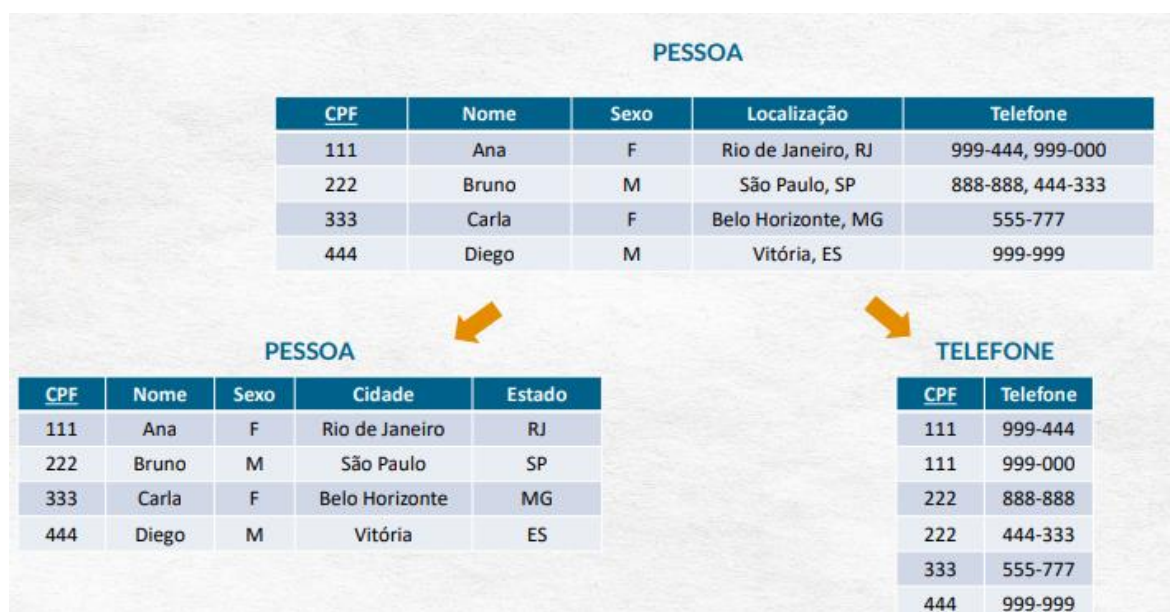


Fonte: Hashtag Programação (2022)

Conforme ilustrado na figura, existem diversas formas normais descritas na literatura, como a Primeira, Segunda e Terceira Forma Normal, além de extensões como Boyce-Codd, Quarta e Quinta Forma Normal. No entanto, na prática, diversos autores consideram que a aplicação até a Terceira Forma Normal (3FN) já é suficiente para garantir um banco de dados bem estruturado, livre da maioria das redundâncias e inconsistências.

A Primeira Forma Normal (1FN) tem como objetivo garantir que todos os atributos de uma tabela sejam atômicos, ou seja, que cada campo armazene apenas um único valor. Isso implica eliminar atributos multivalorados e atributos compostos. Um exemplo comum ocorre quando uma tabela possui um campo que armazena múltiplos telefones em uma única célula ou um campo de endereço contendo diversas informações juntas, como cidade e estado.

Figura 9 – Adequação à Primeira Forma Normal

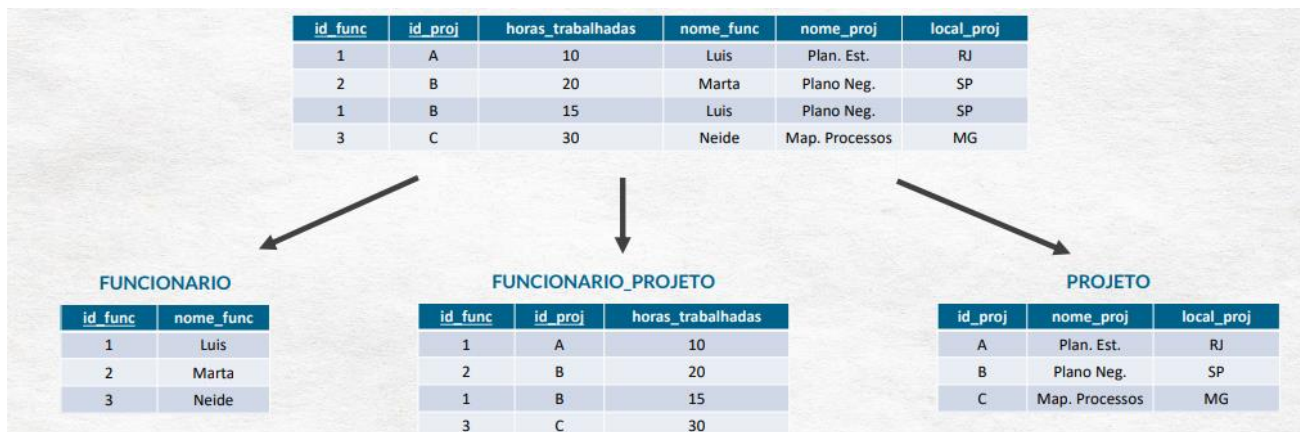


Fonte: Hashtag Programação (2022)

Como observado na figura, os dados originalmente concentrados em uma única tabela são reorganizados, separando atributos multivalorados em uma nova tabela e desmembrando atributos compostos em colunas menores. Esse processo melhora a organização dos dados e facilita futuras consultas.

A Segunda Forma Normal (2FN) busca eliminar dependências funcionais parciais, ou seja, situações em que atributos não chave dependem apenas de parte de uma chave primária composta. Esse problema ocorre quando uma tabela utiliza mais de um campo como identificador e algumas informações estão relacionadas apenas a um desses campos.

Figura 10 – Adequação à Segunda Forma Normal (2FN)



Fonte: Hashtag Programação (2022)

Nesse caso, a solução consiste em dividir a tabela original em novas tabelas, separando os dados de acordo com suas dependências. Dessa forma, cada informação passa a depender corretamente da chave primária completa, reduzindo redundâncias e melhorando a integridade dos dados.

A Terceira Forma Normal (3FN), por sua vez, tem como objetivo eliminar dependências funcionais transitivas. Esse tipo de dependência ocorre quando um atributo não chave depende de outro atributo que também não é chave, e não diretamente da chave primária. Esse cenário indica que a estrutura da tabela pode ser melhorada.

Figura 11 – Adequação à Terceira Forma Normal (3FN)



Fonte: Hashtag Programação (2022)

Conforme demonstrado, a solução consiste em separar esses atributos em uma nova tabela, garantindo que todos os dados estejam diretamente relacionados à chave primária correta. Esse processo evita duplicidade de informações e facilita a manutenção do banco de dados. No contexto da Engenharia de Produção, a aplicação da normalização é essencial para garantir a confiabilidade dos dados utilizados nas análises. Em sistemas como ERPs, que armazenam informações de produção, estoque, pedidos e clientes, uma estrutura bem

normalizada permite realizar consultas mais precisas, reduzir erros e melhorar o desempenho das análises.

Além disso, a normalização contribui diretamente para a eficiência dos sistemas de informação, pois reduz a redundância, melhora a organização dos dados e facilita a atualização das informações. Dessa forma, o engenheiro de produção passa a trabalhar com dados mais consistentes, aumentando a qualidade das decisões tomadas a partir dessas informações.

Portanto, a normalização não é apenas um conceito técnico de banco de dados, mas uma prática essencial para garantir que os dados utilizados na gestão sejam confiáveis, organizados e adequados para análise, reforçando sua importância no contexto da Engenharia de Produção.

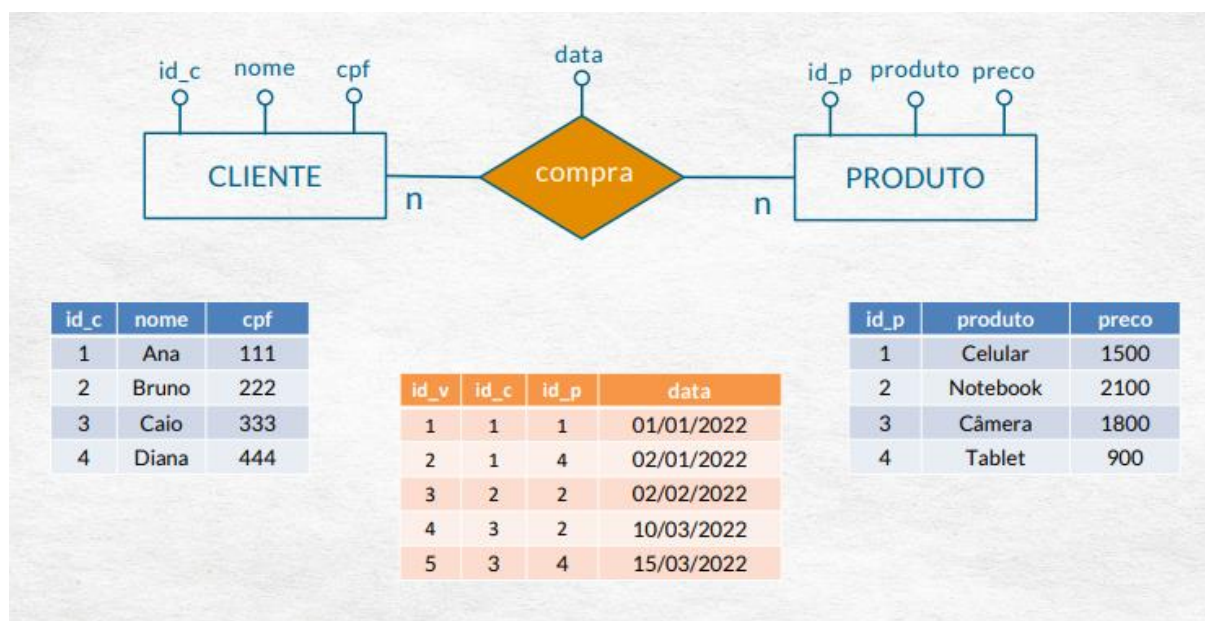
2.6 Agregação e Composição no Modelo Entidade-Relacionamento

No processo de modelagem de dados, além da definição das entidades e dos relacionamentos, existem conceitos mais avançados que permitem representar situações mais complexas do mundo real. Entre esses conceitos, destacam-se a agregação e a composição, que possibilitam estruturar relações de forma mais organizada e sem perda de significado dos dados. O Modelo Entidade-Relacionamento (MER) tem como objetivo representar os elementos de um negócio e a forma como eles se relacionam entre si, servindo como base para a construção do banco de dados.

Dentro desse modelo, os relacionamentos normalmente são representados como associações diretas entre entidades. No entanto, em algumas situações, é necessário tratar um relacionamento como se fosse uma entidade, permitindo que ele participe de outras relações. Esse conceito é conhecido como agregação.

A agregação ocorre quando um relacionamento passa a ser tratado como uma entidade de nível superior, possibilitando a criação de novas relações a partir dele. Esse recurso é especialmente útil quando se deseja adicionar atributos ao próprio relacionamento ou quando ele precisa se conectar a outras entidades no modelo.

Figura 12 – Exemplo de Relacionamento



Fonte: HashTag Programação (2022)

Conforme ilustrado na figura, o relacionamento “compra” entre as entidades cliente e produto não se limita apenas à ligação entre esses elementos, mas passa a representar uma nova estrutura de dados. Nesse caso, a relação de compra pode conter atributos próprios, como data da compra, quantidade adquirida ou valor total, deixando de ser apenas uma conexão e passando a ter significado próprio dentro do sistema.

Na prática, isso significa que a relação entre cliente e produto é transformada em uma nova entidade intermediária, frequentemente chamada de tabela associativa. Essa estrutura é comum em relacionamentos do tipo muitos para muitos (N:N), nos quais um cliente pode comprar vários produtos e um produto pode ser comprado por vários clientes. A tabela intermediária armazena essas conexões e permite registrar informações adicionais sobre cada ocorrência da relação.

Já o conceito de composição está relacionado à ideia de dependência entre entidades, indicando que um elemento só existe em função de outro. Em termos práticos, a composição representa uma relação forte entre entidades, na qual a existência de uma está diretamente vinculada à existência da outra.

Por exemplo, em um sistema de produção, pode-se considerar que uma ordem de produção é composta por várias etapas ou operações. Essas etapas não possuem sentido isoladamente, pois dependem da existência da ordem principal. Caso a ordem seja excluída, todas as suas etapas também deixam de existir. Esse tipo de relação caracteriza a composição, pois há uma dependência direta entre os elementos.

A distinção entre agregação e composição é importante para a modelagem adequada dos dados. Enquanto a agregação permite tratar relacionamentos como entidades independentes, ampliando a flexibilidade do modelo, a composição representa vínculos mais rígidos, nos quais os elementos dependem estruturalmente uns dos outros.

No contexto da Engenharia de Produção, esses conceitos são amplamente aplicáveis, especialmente em sistemas que envolvem múltiplas relações entre processos, produtos e recursos. A correta utilização da agregação permite representar operações complexas, como vendas, produção e movimentações de estoque, enquanto a composição auxilia na organização hierárquica de elementos dependentes, como ordens de produção e suas respectivas etapas.

Dessa forma, a utilização desses conceitos contribui para a construção de modelos de dados mais fiéis à realidade operacional, garantindo maior clareza, organização e capacidade de análise das informações. Isso reforça a importância do domínio do Modelo Entidade-Relacionamento como base para o desenvolvimento de bancos de dados eficientes e alinhados às necessidades da Engenharia de Produção.

2.7 Aplicação prática utilizando SQL Server e Python

Com o objetivo de aplicar na prática os conceitos relacionados à extração e análise de dados apresentados ao longo deste trabalho, será utilizado o banco de dados SQL Server pertencente ao ambiente industrial da empresa. O SQL Server foi escolhido devido à sua ampla utilização em sistemas ERP corporativos e à grande capacidade de armazenamento e gerenciamento de dados relacionais utilizados nos processos produtivos.

No contexto da Engenharia de Produção, o acesso direto ao banco de dados permite maior autonomia analítica na obtenção de informações relacionadas à produção, estoque, ordens de fabricação, consumo de matéria-prima, tempos operacionais e indicadores de desempenho. Dessa forma, o engenheiro de produção passa a depender menos de relatórios previamente estruturados, podendo construir análises específicas de acordo com a necessidade operacional. Para realização da extração dos dados será utilizada a linguagem Python, devido à sua ampla aplicação em automação, ciência de dados e integração com bancos de dados relacionais. Além disso, o Python apresenta elevada compatibilidade com bibliotecas voltadas para manipulação e tratamento de dados, permitindo maior flexibilidade no desenvolvimento das análises.

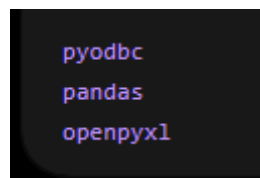
A conexão entre o Python e o SQL Server será realizada por meio da biblioteca pyodbc, responsável pela comunicação entre a aplicação e o banco de dados. Após o estabelecimento

da conexão, serão executadas consultas em SQL para recuperação das informações desejadas diretamente nas tabelas do sistema.

Posteriormente, os dados extraídos serão organizados e tratados utilizando a biblioteca pandas, permitindo operações como filtragem, agrupamento, limpeza de dados e geração de relatórios analíticos. Após o processamento das informações, os resultados poderão ser exportados para arquivos em formatos como Excel (.xlsx) ou CSV, facilitando a utilização dos dados em dashboards, indicadores gerenciais e análises aplicadas ao PCP e aos processos produtivos.

Dessa forma, a utilização integrada entre SQL Server e Python demonstra, na prática, como ferramentas computacionais podem ampliar a capacidade analítica do engenheiro de produção, contribuindo para maior agilidade, autonomia e precisão na tomada de decisão organizacional.

Figura 13 – Bibliotecas



Fonte: Elaborado pelo Autor (2026)

Figura 14 – Conexão SQL Server

```
import pyodbc
import pandas as pd

conexao = pyodbc.connect(
    'DRIVER={SQL Server};'
    'SERVER=SERVIDOR;'
    'DATABASE=BANCO;'
    'UID=USUARIO;'
    'PWD=SENHA'
)
```

Fonte: Elaborado pelo Autor (2026)

REFERÊNCIAS

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. *Software Architecture in Practice*. 4. ed. Boston: Addison-Wesley, 2021.

CODD, Edgar F. A relational model of data for large shared data banks. *Communications of the ACM*, v. 13, n. 6, p. 377–387, 1970.

DATE, C. J. *An Introduction to Database Systems*. 8. ed. Boston: Addison-Wesley, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. *Fundamentals of Database Systems*. 7. ed. Boston: Pearson, 2018.

FOWLER, Martin. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Boston: Addison-Wesley, 2012.

LAUDON, Kenneth C.; LAUDON, Jane P. *Management Information Systems: Managing the Digital Firm*. 16. ed. Harlow: Pearson, 2021.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Software Engineering: A Practitioner's Approach*. 9. ed. New York: McGraw-Hill, 2020.

SADALAGE, Pramod J.; FOWLER, Martin. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Boston: Addison-Wesley, 2013.

SENGE, Peter M. *A Quinta Disciplina: Arte e Prática da Organização que Aprende*. Rio de Janeiro: BestSeller, 2017.

TURBAN, Efraim et al. *Information Technology for Management: On-Demand Strategies for Performance, Growth and Sustainability*. 11. ed. Hoboken: Wiley, 2020.